OS Lab Project – Huffman Coding File Compressor (MiniZip)

Aayush Mishra (Roll No. 2025UCS1706)
Branch: Computer Science and Engineering – CSE, Section: [Section]
Course: Operating Systems (COCSC301)
Project Repository: [GitHub link, if any]

Contents: 1. Problem Statement | 2. Operating System Concepts Used | 3. System Modules and their Purpose | 4. File Operations and System Calls | 5. Test Results, Features and How to Run

I. PROBLEM STATEMENT

Every file on disk is, to the operating system, just a sequence of bytes. Many of those bytes are redundant: in a text file the letter "e" or a space occurs far more often than a rarely used symbol, yet a normal file spends 8 bits on each. Compression utilities such as zip and gzip exploit this to save disk space and transfer time. In lab courses, however, file handling is usually taught through small programs that open a file, print it and close it, without showing how a real system utility reads a whole file, processes it, and writes a new file back to persistent storage through the kernel.

This project is a command-line file compressor and decompressor written in C++17 using only the standard library (zero third-party dependencies). It applies **Huffman coding**, a lossless algorithm that assigns shorter bit codes to frequent bytes and longer codes to rare bytes. The program works on *any* file type (text, source code, binary) because it treats input as raw bytes. It is a good fit for an Operating Systems lab because it exercises file descriptors and system calls, buffered binary I/O, file offsets and rewinding, dynamic memory management, and process exit status, all inside one small but genuinely useful system program.

A. Objectives

- **Lossless Compression:** Reduce file size using Huffman coding such that the decompressed file is byte-for-byte identical to the original.
- **Binary-Safe File I/O:** Open files in binary mode so that no newline translation occurs and any file type can be processed.
- **Two-Pass File Processing:** Pass 1 reads the file to count byte frequencies; the file offset is then rewound and Pass 2 reads it again to encode.
- **Bit-Level Output:** Pack variable-length codes into whole bytes with a BitWriter, because the OS can only read and write in units of bytes.
- **Self-Describing Archive:** Store a header (frequency table and original size) so that the decompressor can rebuild the identical tree with no other input.
- **Memory Discipline:** Allocate tree nodes on the heap and release every node explicitly (freeTree) to prevent memory leaks.
- **Defensive Behaviour:** Detect missing files, unwritable outputs, empty input and corrupted headers and report them clearly with a non-zero exit status.

II. OPERATING SYSTEM CONCEPTS USED

The project demonstrates several fundamental Operating System concepts in practice. Table I shows how each one appears in the code.

TABLE I
OPERATING SYSTEM CONCEPTS USED

Core OS Concept	How it is Used in the Project	Underlying Mechanism
System Programs & CLI	Runs as a command-line utility. Mode and file names arrive from the shell as arguments; success or failure is returned as the exit status.	main(argc, argv), return 0 / 1, exit(1)
System Calls & Kernel Invocation	Opening, reading, writing, seeking and closing files all request service from the kernel; the program never touches disk sectors itself.	open(), read(), write(), lseek(), close() (Windows: CreateFileW, ReadFile, WriteFile, SetFilePointerEx)
Files as Byte Streams	Input and output are handled as raw byte streams with no interpretation of the content.	std::ios::binary
File Offset & Random Access	After Pass 1 reaches end-of-file, the EOF flag is cleared and the offset is moved back to byte 0 for Pass 2.	in.clear(); in.seekg(0) -> lseek(fd, 0, SEEK_SET)
Buffered I/O	Codes are written one byte at a time, but the C++ stream collects them in a user-space buffer and hands the kernel large blocks, so the number of system calls is much smaller than the number of bytes.	std::filebuf buffer; BitWriter
Memory Management	Tree nodes are created on the heap while building the tree and all of them are freed after use. Memory use depends on the number of distinct bytes (at most 256 leaves), not on file size.	new / delete (malloc: brk / mmap)
Process Termination	Fatal errors call exit(1) so that the shell or a script can detect failure.	exit status
Error Handling & Protection	Failure to open a file (missing file, permission denied) and short header reads are detected and reported without a crash.	is_open(), stream state (!in)
Data Structures in System Software	A min-heap (priority queue) selects the two rarest symbols; a binary tree gives the prefix-free codes.	std::priority_queue, Node tree

III. SYSTEM MODULES AND THEIR PURPOSE

The whole program is contained in one source file, minizip.cpp, divided into clearly separated parts. Table II lists them. Compression logic (tree, codes) is kept apart from file handling (compressFile, decompressFile) and from bit packing (BitWriter).

TABLE II
SYSTEM MODULES AND THEIR PURPOSE

Component	Role and Purpose	Key Functions / Members
struct Node	One node of the Huffman tree: byte value, frequency and two child pointers. Internal nodes use the dummy byte "$".	data, freq, left, right
struct CompareNode	Comparator that makes std::priority_queue a min-heap, so the lowest frequency is popped first.	operator()
buildHuffmanTree()	Creates one leaf per byte with non-zero frequency, then repeatedly merges the two smallest nodes until one root remains. Handles the single-unique-byte edge case.	pq.push(), pq.pop()
generateCodes()	Recursive traversal of the tree. Left edge adds "0", right edge adds "1"; at each leaf the path string is saved in a 256-entry table.	codes[256]
freeTree()	Post-order deletion of every node to avoid leaks.	delete root

Component	Role and Purpose	Key Functions / Members
class BitWriter	Collects single bits into an 8-bit buffer (most significant bit first), writes a byte when full, and pads the last partial byte with zeros.	writeBit(), flush()
compressFile()	Complete compression pipeline: count, build, write header, encode.	ifstream, ofstream
decompressFile()	Reads header, rebuilds the same tree, then walks the tree bit by bit to restore bytes.	read(), put()
main()	Validates arguments (-c or -d) and dispatches to compress or decompress.	argc, argv

A. Flow of Compression

1. Open and count: the input file is opened in binary mode and every byte is read once to fill freq[256]; the total size is saved. An empty file stops here.

2. Build tree: a leaf node is pushed into the min-heap for each byte that occurs; the two smallest are merged repeatedly to form the Huffman tree.

3. Generate codes: a recursive walk fills the lookup table, so frequent bytes get short codes.

4. Write header: the output file is created and the 1024-byte frequency table and the 4-byte original size are written.

5. Encode: the input is rewound and read again; the code of each byte is pushed bit by bit into BitWriter, which flushes whole bytes to the output file.

6. Finish: the last byte is padded, the sizes are reported, and the tree is freed. Closing of both files happens automatically when the stream objects go out of scope.

B. Flow of Decompression

1. Read and validate the 1028-byte header. **2.** Rebuild the identical tree from the frequency table. **3.** Read the data bytes, test each bit from bit 7 down to bit 0, and move left on 0 or right on 1. **4.** On reaching a leaf, write its byte and return to the root. **5.** Stop after exactly totalBytes bytes have been produced, which makes the padding bits in the final byte harmless.

C. Archive (.huff) File Format

TABLE III
LAYOUT OF THE COMPRESSED FILE

Offset	Size	Field
0	1024 bytes	freq[0..255], 256 x uint32_t (frequency of each byte value)
1024	4 bytes	totalBytes, uint32_t (original file size)
1028	variable	Huffman-coded bit stream, MSB first; last byte zero-padded

IV. FILE OPERATIONS AND SYSTEM CALLS

User programs do not write to disk sectors directly. They request the kernel to do it through system calls. In this project the C++ stream classes wrap those calls. Table IV shows the usual mapping on Linux (glibc / libstdc++); on Windows the same stream operations end up in the Win32 calls named in Table I. The program does not invoke the system calls by name itself, so this mapping describes what the standard library normally does on its behalf.

TABLE IV
MAPPING OF PROGRAM OPERATIONS TO SYSTEM CALLS

Operation	C++ Code	System Call	What the OS Does
Open input	ifstream(path, binary)	open(O_RDONLY)	Resolves the path, checks permissions, returns a file descriptor.
Read data	in.get(ch)	read()	Copies bytes from the page cache (or disk) in blocks into the stream buffer.
Rewind	in.clear(); in.seekg(0)	lseek(fd, 0, SEEK_SET)	Resets the file offset to the start; no data is moved.
Create output	ofstream(path, binary)	open(O_WRONLY|O_CREAT|O_TRUNC)	Creates the file or truncates an existing one to 0 bytes.
Write header / data	out.write(), out.put()	write()	Copies buffered bytes into the kernel page cache; the disk is updated later.
Get size	out.tellp()	lseek(fd, 0, SEEK_CUR)	Returns the current offset, which equals the bytes written.
Close files	destructors at scope end	close()	Flushes buffers and releases the file descriptor.

A. Defensive Error Handling

The program checks every step that depends on the outside world and exits with a clear message instead of crashing.

TABLE V
DEFENSIVE ERROR HANDLING

Condition	How it is Detected	Response
Input file missing or unreadable (ENOENT / EACCES)	is_open() returns false	Error message, exit(1)
Output file cannot be created	is_open() returns false	Frees the tree, error message, exit(1)
Empty input file	totalBytes == 0	Message; no archive is created
Corrupted or truncated archive	Stream state false after reading header	"Invalid or corrupted header", exit(1)
Wrong number of arguments	argc != 4	Prints usage, returns 1
Unknown mode flag	Not -c and not -d	Error message, returns 1

V. TEST RESULTS, FEATURES AND HOW TO RUN

A. Test Results

The program was compiled with g++ -std=c++17 -O2 and tested on the inputs below. In every case the file produced by decompression was compared with the original using cmp and was **identical**.

TABLE VI
COMPRESSION RESULTS ON SAMPLE INPUTS

Input	Original (B)	Archive (B)	Archive / Original
English-like text	65,200	38,612	59.2 %
C++ source (minizip.cpp)	7,130	5,373	75.4 %
5000 x the same letter	5,000	1,653	33.1 %
Random binary data	50,000	51,028	102.1 %
5-byte text file	5	1,030	20,600 %

Observations. (1) Text with a skewed byte distribution compresses well. (2) Random data has an almost uniform distribution, so Huffman coding cannot shorten it and the fixed 1028-byte header makes the archive slightly larger. (3) For very small files the header dominates and the archive is much larger than the input. (4) A file made of one repeated byte still needs 1 bit per byte because of the edge-case handling, giving about 8:1 on the data part. These results match the theory: Huffman coding helps only when byte frequencies are uneven and the file is larger than the header.

B. Key Features

- **Lossless and Universal:** works on any file type and restores it exactly.
- **Two-Pass Design with File Rewind:** demonstrates file offsets and lseek in practice.

- **Custom Bit Writer:** packs arbitrary-length codes into bytes with correct padding.
- **Self-Contained Archive:** the header carries everything needed to decode.
- **Memory Safe:** every node is freed; memory use is bounded by 256 symbols, not file size.
- **Clear Diagnostics:** reports sizes and errors; returns a non-zero status on failure.

C. How to Compile and Run

The project needs only a C++17 compiler (g++, clang++ or MSVC).

```
# 1. Compile
g++ -std=c++17 -O2 -o minizip minizip.cpp

# 2. Compress
./minizip -c input.txt output.huff

# 3. Decompress
./minizip -d output.huff restored.txt

# 4. Verify (prints nothing if identical)
cmp input.txt restored.txt
```

D. Limitations and Future Work

- The fixed 1028-byte header is wasteful for small files; storing only code lengths (canonical Huffman) would shrink it.
- 32-bit counters limit input to 4 GiB; the archive has no magic number or checksum, so a wrong or damaged file is not reliably detected.
- The tree is rebuilt with std::priority_queue, so equal-frequency tie-breaking must match between compressor and decompressor; an archive is safest to decode with the same build.
- Possible extensions: a progress display, multi-file archives, multithreaded block compression, and a graphical front end.

E. Summary

This project connects abstract Operating System theory with a practical system utility. By building a Huffman compressor from scratch, it shows how a user program asks the kernel for file descriptors, reads and writes through buffered system calls, repositions the file offset, manages heap memory, and reports success or failure through exit status, while achieving real, measurable savings on suitable data.